

Analyse des Algorithmes de chiffrements

Pourquoi ECDSA est l'algorithme de signature le plus présent dans le domaine bancaire ? Voici une étude des autres algorithmes présents sur le marché :

Algorithme	Type	Rôle principal	Statut Marché
AES (256 bits)	Symétrique	Chiffrement des données (bases de données, fichiers).	Standard absolu. Incontournable pour la performance.
RSA (3072+ bits)	Asymétrique	Signatures et échange de clés (vieux systèmes).	Legacy. En phase de retrait car trop lourd.
ECDSA (256 bits)	Asymétrique	Authentification et intégrité des transactions.	Standard moderne. Utilisé par les banques mobiles et la blockchain.
EdDSA (Ed25519)	Asymétrique	Signature haute performance.	Émergent. Plus rapide et plus sûr qu'ECDSA contre certaines attaques.

AES est un algorithme de chiffrement symétrique, on ne peut pas l'utiliser pour faire de la signature cryptographique. En effet, comme il utilise la même clé pour chiffrer et déchiffrer les messages, on ne peut être sûr que ce soit bien l'auteur du message qui l'ait signé.

Nous allons donc se concentrer sur RSA et ECDSA, EdDSA étant une variante de ECDSA. Il sera prévu de faire une analyse chiffrée entre les deux algorithmes pendant le projet.

État de l'Art – RSA et ECDSA

Le protocole « Rivest Shamir Adleman », ou RSA, est un système de chiffrement asymétrique créé en 1977 aux États-Unis par par Ronald Rivest, Adi Shamir et Leonard Adleman. Il repose sur une paire de clés, l'une publique et l'autre privée, qui sont générées par la multiplication de deux nombres premiers. La clé publique est utilisée pour chiffrer le message, et la clé privée sert à le déchiffrer sur l'appareil du destinataire. Les deux clés sont générées par la même personne, qui diffuse sa clé publique à ses correspondants voulant communiquer avec lui. Ce protocole repose sa sécurité sur son chiffrement dit « calculatoirement indéchiffrable » si l'attaquant ne possède que la clé publique de la paire de clé.

Pour comprendre son chiffrement, il faut comprendre comment les clés encryptent et décryptent les messages. La clé publique est composée de deux informations distinctes : l'exposant d'encryptage, noté « e » et le module, noté « n ». La clé privée est un nombre noté « d » créée à partir de « e » et « n ». Nous pouvons alors définir les deux clés :

$$k_{pub} = (n, e) \text{ et } k_{pr} = d.$$

On définit « x » le message à encrypter tel que « $x \in \mathbb{Z}_n$ » ou $\mathbb{Z}_n$ correspond à l'anneau des entiers modulo n . On définit « y » le message encrypté tel que « $y \in \mathbb{Z}_n$ ». On peut alors définir l'encryptage tel que :

$$(formule\ d'encryptage)\ y = e_{k_{pub}}(x) = x^e \equiv n$$

On peut alors définir le décryptage par la formule ci-dessous comme la fonction inverse de la formule d'encryptage par le lien entre « d », « e » et « n » provenant du théorème d'Euler qui sera admis pour cette explication, tel que :

$$(formule\ de\ décryptage)\ x = d_{k_{pr}}(y) = y^d \equiv n$$

Nous allons maintenant voir comment les clés sont générées. Etant asymétrique, la clé publique et la clé privée noté « k_{pub} » et « k_{pr} » sont créés en même temps et dépendent des mêmes paramètres. On choisit d'abord deux nombres premiers, de préférence très grand, noté « p » et « q ». On en déduit alors le module « n », étant le produit de ces derniers. On définit alors l'indicatrice d'Euler notée « $\phi(n)$ » tel que :

$$(formule\ de\ l'indicatrice\ d'Euler)\ \phi(n) = (p - 1) * (q - 1)$$

On peut ainsi déterminer l'exposant d'encryptage « e », tel qu'il doit être premier avec l'indicatrice d'Euler. Cette condition se vérifie si le plus grand diviseur commun entre les deux nombres est un, tel que $\gcd(e, \phi(n)) = 1$. Il ne reste alors plus qu'à déterminer l'inverse modulaire de « e » modulo $\phi(n)$ notée « d » à l'aide de l'algorithme d'Euclide étendu tel qu'il vérifie l'équation suivante :

$$(formule\ de\ la\ vérification\ de\ l'inverse\ modulaire)\ d * e \equiv 1 \mod \phi(n)$$

Nous avons donc vu que ce système de chiffrement repose sur une paire de clés définies par $k_{pub} = (n, e)$ et $k_{pr} = d$. Ces clés sont créées à partir du produit de nombres premiers et sont donc pratiquement indéchiffrables car la clé de déchiffrement privée est pratiquement impossible à obtenir depuis la clé de chiffrement.

Le protocole ECDSA est une variante de l'algorithme DSA (Digital Signature Algorithm) utilisant la cryptographie sur les courbes elliptiques (ECC). Proposé pour la première fois par Scott Vanstone en 1992, il est devenu un standard international (ISO, ANSI, NIST) au début des années 2000. Contrairement à RSA qui repose sur la difficulté de la factorisation d'entiers, ECDSA fonde sa sécurité sur la complexité du problème du logarithme discret sur les courbes elliptiques (ECDLP).

En effet, on définit E une courbe elliptique définie tel que :

$$y^2 = x^3 + ax + b \pmod{p}$$

Avec p un nombre premier et $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$.

Les points de la courbe, accompagnés d'un "point à l'infini" noté O , forment un groupe abélien sous l'opération d'addition de points. Cette opération géométrique (loi "corde-et-tangente") permet de définir la multiplication scalaire :

$$Q = k * G$$

L'ECDLP stipule qu'étant donnés deux points G et Q , il est "calculatoirement impossible" de retrouver l'entier k tel que $Q = k * G$ si k est suffisamment grand. Avant toute transaction, les parties doivent s'accorder sur les paramètres du domaine :

p : Le module premier définissant la taille du corps.

a, b : Les coefficients de l'équation de la courbe.

G : Le point de base (générateur) choisi sur la courbe.

n : L'ordre du point G (le plus petit entier tel que $G * n = O$.)

La génération de la paire de clés s'effectue comme suit :

1. On choisit aléatoirement un entier d dans l'intervalle $[1, n-1]$. Cet entier constitue la clé privée.
2. On calcule le point $Q = d \times G$. Ce point constitue la clé publique.

Lorsqu'une transaction bancaire doit être signée, l'expéditeur suit les étapes suivantes :

1. Calcul du condensé (hash) du message M avec un algorithme comme SHA-256 : $e = H(M)$
2. Sélection d'un nombre entier aléatoire k (appelé nonce) tel que $1 < k < n-1$.
3. Calcul du point de la courbe $(x, y) = k \times G$.
4. Calcul de la première composante de la signature : $r = x \pmod n$. Si $r = 0$, on recommence au point 2.
5. Calcul de la seconde composante $s = k^{-1}(e + r \times d)$. Si $s = 0$, on recommence au point 2.

La signature transmise avec la transaction est le couple (r, s) .

Pour valider la transaction, la banque (ou le destinataire) possède la clé publique Q, le message M et la signature (r, s) . Le processus est le suivant :

1. Vérifier que r et s sont bien dans l'intervalle $[1, n-1]$.
2. Calculer $e = H(M)$.
3. Calculer l'inverse modulaire w de s mod n.
4. Calculer les deux scalaires $u1 = e \times w \pmod n$ et $u2 = r \times w \pmod n$.
5. Calculer le point $X = u1 \times G + u2 \times Q$.
6. La signature est valide si $r \equiv x0 \pmod n$

Une alternative performante aux courbes de Weierstrass classiques réside dans l'utilisation des courbes d'Edwards (notamment les formes "tordues" comme Ed25519). Leur avantage majeur repose sur des formules d'addition complètes et unifiées, valables pour tous les points de la courbe sans exception géométrique. Nous les utiliserons lors de ce projet.